

Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Davide Fissore  

Université Côte d’Azur, Inria, France

Enrico Tassi  

Université Côte d’Azur, Inria, France

Abstract

We mechanize two operational semantics for PROLOG with cut and prove them equivalent, in the Rocq prover. The first semantics closely models the ELPI’s implementation, it is based on a stack of choice points and is well suited for studying optimizations employed by PROLOG abstract machines. The second semantics is instead based on and-or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics (1) to prove properties about the effect of the cut operator in the evaluation of choice points, since the rules from classical logic do not apply, and (2) to reason about the determinacy analysis introduced in Elpi version 3.0, which statically identifies computations that produce at most one result.

2012 ACM Subject Classification Theory of computation → Operational semantics

Keywords and phrases Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Davide Fissore*: This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the “UCA DS4H” project, reference ANR-17-EURE-0004.

1 Introduction

ELPI is a dialect of λ PROLOG (see [16, 17, 7, 13]) used as an extension language for the ROCQ prover (formerly the COQ proof assistant). ELPI has become an important infrastructure component: several projects and libraries depend on it [14, 3, 4, 22, 8, 9]. Other remarkable libraries include the Hierarchy-Builder library-structuring tool [5] and Derive [20, 21, 12], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, ELPI is equipped with a static analysis for determinacy [10] to help users control backtracking. ROCQ users are familiar with functional programming, but not necessarily with logic programming; in particular, uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker allows the user to assert which predicates should be considered deterministic and then checks that these predicates behave like functions, i.e., they commit to their first solution and leave no *choice points* (places where backtracking could resume). A choice point correspond to an suspended *alternative* in the execution trace. In the following we use *choice point* and *alternative* as synonyms.

This paper reports our first steps towards a mechanization, in the ROCQ prover, of the determinacy analysis from [10]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for PROLOG with cut. The first is a stack-based semantics that closely models ELPI’s implementation and is similar to the semantics mechanized by Pusch in ISABELLE/HOL [18] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations



© Jane Open Access and Joan R. Public;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:17

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

```

func f int -> int. % f is a deterministic predicate. The two rules
f 1 2.             % have non-verlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int. % r is a non-deterministic predicate. The heads
r 0 0.             % overlap: the query for 0 has three applicable
r 0 1.             % rules.
r 0 2 :- !.
func g int -> int. % g is deterministic: if the first rule succeeds, the
g A C :- r A B, f B C, !. % second one is cut away, as well as the choice
g D F :- f D E, f E F.   % points left by the call to r.

```

■ **Figure 1** Running example of a ELPI program

used by standard PROLOG abstract machines [23, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by cut are explicit and we prove the two semantics equivalent. Using the tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive queries. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1 .

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis [10]. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. Firstly, at most one rule should be used on a given query. This condition is satisfied if (i) the rules have non-overlapping heads: if two heads do not overlap in the inputs they accept, then no query can unify with both, or if (ii) we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded. Secondly, each rule of a deterministic predicate should (i) either call functions, this ensure that no choice point is left after the execution of the premises of the chosen rule, (ii) or call relations provided that they are followed by the cut operator, so that any allocated choice points preceeding the cut are discarded.

We show that if every rule defining a predicate passes this analysis, then any call to that predicate leaves no choice points.

2 Preliminaries: syntax and informal semantics of cut

Both semantics share the same notion of program, written $\mathbb{P}$. Figure 1 is our running example. A program consists of a list of rules R together with a signature environment sigT . Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, and whether the predicate is deterministic. Inputs come before outputs.

A rule consists of a head term T_m and a list of premises. Each premise is an Atom , i.e. either the cut operator or a predicate call. Terms T_m include predicate symbols, data, and

```

Inductive Tm := ...
| Pred of P | Var of V.
Inductive sig := NonDet | Det.
Definition sigT :=
  P -> sig * nat * nat.

```

■ **Figure 2** Syntax of ELPI programs

```

Inductive A := cut | call : Tm -> A.
Record R := { head : Tm; premises : list A }.
Record P := { rules : list R; sig : sigT }.
Definition Σ := {fmap V -> Tm}.

```

■ **Figure 3** Syntax of ELPI programs

variables. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ -calculus style (no parentheses).

The set of variables $\mathbf{V}$ is countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the `finmap` library [15]. The empty substitution is written ε while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics.

Definition `bc` : $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \mathbf{Tm} \rightarrow \Sigma \rightarrow \mathcal{F}_v * \mathbf{list} (\Sigma * \mathbf{list} \mathbf{A}) :=$

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_v$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization we abstract the unifier and its properties, namely

```
Record Unif := {
  unify : Tm → Tm → Σ → option Σ;
  matching : Tm → Tm → Σ → option Σ;
}.
```

TODO: state axioms and explain matching; cite Dale's unification results. In this paper we assume the reader is familiar with first-order unification.

2.1 The cut operator

The cut operator in ELPI implements the *hard cut* (see, e.g., the documentation of SWI-PROLOG [19]). Whenever the backchaining function `bc` returns a list of alternatives, the first one is explored immediately, while the others are suspended as *choice points*. Within a given alternative, premises are processed from left to right. When a cut is encountered, it discards (i) all choice points created by `bc` for the current call and (ii) all choice points created while executing the premises preceding the cut.

For example, consider the program P in Figure 1 and the query $\mathbf{g} \ 0 \ \mathbf{R}$. Both rules for $\mathbf{g}$ apply to this query. Backchaining therefore returns the following two alternatives:

$[(\sigma_1, [\mathbf{r} \ \mathbf{A} \ \mathbf{B}, \ \mathbf{f} \ \mathbf{B} \ \mathbf{C}, \ !]), (\sigma_2, [\mathbf{f} \ \mathbf{D} \ \mathbf{E}, \ \mathbf{f} \ \mathbf{E} \ \mathbf{F}])]$

with $\sigma_1 := \{\mathbf{A} \mapsto 0, \mathbf{C} \mapsto \mathbf{R}\}$ and $\sigma_2 := \{\mathbf{D} \mapsto 0, \mathbf{F} \mapsto \mathbf{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_v$ any further.

Execution continues with the first premise of the first alternative, namely $\mathbf{r} \ \mathbf{A} \ \mathbf{B}$ under σ_1 , i.e. $\mathbf{r} \ 0 \ \mathbf{B}$. The remaining alternatives are stored as choice points. Backchaining $\mathbf{r} \ 0 \ \mathbf{B}$ returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [!])]$ where $\sigma_3 := \sigma_1 + \{\mathbf{B} \mapsto 0\}$; $\sigma_4 := \sigma_1 + \{\mathbf{B} \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{\mathbf{B} \mapsto 2\}$.

The next step of interpretation continues with $\mathbf{f} \ \mathbf{B} \ \mathbf{C}$ under σ_3 , that is, $\mathbf{f} \ 0 \ \mathbf{R}$, and leaves $[(\sigma_4, []), (\sigma_5, [!])]$ as new choice points. This time, backchaining fails (no rule for $\mathbf{f}$ matches input 0). We therefore backtrack to the most recently introduced choice point and retry $\mathbf{f} \ \mathbf{B} \ \mathbf{C}$ under σ_4 , i.e. $\mathbf{f} \ 1 \ \mathbf{R}$. This succeeds with $\sigma_6 := \sigma_4 + \{\mathbf{R} \mapsto 2\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for $\mathbf{r}$ and one from the second rule for $\mathbf{g}$ (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for $\mathbf{g}$ was selected.

```

Inductive tree :=
  | KO | OK                                     (* failure and success *)
  | Todo of Atom                               (* unexplored branch *)
  | Or of option tree &  $\Sigma$  & tree             (*  $A \vee B_\sigma$  *)
  | And of tree & list Atom & tree.            (*  $A \wedge_{B_0} B$  *)

Inductive tag := Expanded | CutBrothers | Failed | Success.
Definition step :  $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow (\mathcal{F}_v * \text{tag} * \text{tree}) :=$ 
Definition prune :  $\mathbb{B} \rightarrow \text{tree} \rightarrow \text{option tree} :=$ 
Inductive runT :  $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow \text{option } (\Sigma * \text{option tree}) \rightarrow \mathbb{B} \rightarrow \mathcal{F}_v \rightarrow \text{Prop} :=$ 

```

■ Figure 4 Signatures of the tree-based semantics

(In contrast, a cut does not discard choice points created earlier; in this example, there are none.)

The final solution is: $\{A \mapsto 0, B \mapsto 1, C \mapsto R, R \mapsto 2\}$.

In Sections 3 and 4, we provide fully mechanized operational semantics for PROLOG with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

Notational conventions In the following section we note $\mathbb{B}$ for the boolean type, and $\top$ and $\perp$ for **true** and **false**. The option instances are noted $\bullet t$ for *Some* t and $\square$ for *None*. Following the Mathematical Components library, on which we depend, we use $\mathbf{x}.1$ and $\mathbf{x}.2$ to denote the first and second projection applied to the pair $\mathbf{x}$.

3 And-Or Tree semantics

An And-Or tree is a stratified, variable-width tree. It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

The tree is built incrementally. The **Todo** node represents an unexplored branch (an **Atom**). When the atom is a **call**, we use the backchaining procedure from Section 2 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the **step** procedure described in Section 3.2.

The Rocq data type for And-Or trees is given in Figure 4. In addition to **Todo**, it provides two distinguished nodes, **KO** and **OK**, which represent respectively the smallest failed and successful tree.

The *Or* node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A . The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ , which becomes the current substitution when backtracking to B .

The *And* node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to be solved after A . We call B_0 the *reset point* for B : it is used to restore the initial state of B when backtracking occurs within A . An example is shown in Section 3.3.

A graphical representation of a tree is shown in Figure 6b. For compactness, we depict *And* and *Or* nodes as n -ary (rather than binary), with children associating to the right. The

```

Fixpoint next  $\sigma$   $t$  :  $\Sigma$  * tree :=
  match  $t$  with
  | Todo _ | KO | OK  $\Rightarrow$  ( $\sigma$ ,  $t$ )
  | Or  $\square$   $\sigma'$  B  $\Rightarrow$  next  $\sigma'$  B
  | Or  $\bullet$ A _  $\Rightarrow$  next  $\sigma$  A
  | And A _ B  $\Rightarrow$ 
    let ( $\sigma'$ , pA) := next  $\sigma$  A in
    if pA == OK then next  $\sigma'$  B
    else ( $\sigma'$ , pA)
  end.

Definition next_subst  $\sigma$   $t$  := (next  $\sigma$   $t$ ).1.
Definition next_tree  $t$  := (next  $\varepsilon$   $t$ ).2.

Definition success  $t$  := next_tree  $t$  == OK.
Definition failed  $t$  := next_tree  $t$  == KO.
Definition incomplete  $t$  :=
  if next_tree  $t$  is Todo _ then  $\top$  else  $\perp$ .

```

■ Figure 5 *next* and derived functions

145 *KO* node acts as the terminating element of an *Or* list, while *OK* is the terminating element
 146 of an *And* list.

147 3.1 The tree-based semantics

148 The tree is constructed incrementally by two recursive functions, *step* and *prune*. Both
 149 traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved
 150 thanks to the *next_tree* (in Figure 5). In the snippet we also identify special kinds of trees:
 151 depending on the *next_tree*, we distinguish between *success*, *failed* and *incomplete* trees.
 152 Since all functions must terminate in Rocq, we define their transitive closure as the inductive
 153 relation *runT*.

154 Intuitively, *runT* iterates calls to *step* on incomplete trees to evaluate the *Todo* node.
 155 Upon failed trees, it calls *prune* to find the next alternative and continues from there, if one
 156 exists. We detail *step*, *prune*, and *runT* in Sections 3.2–3.4.

157 In Figure 6 we mark with a black arrow the result of *next_tree*.

158 3.2 The *step* procedure

159 The *step* procedure takes as input a program, a set of variables, a substitution, and a tree,
 160 and returns an updated set of variables, a *tag*, and an updated tree.

161 The set of variables is assumed to contain all variables occurring in the tree. It is passed
 162 to backchain so that rules can be refreshed correctly.

163 The *tag* (see Figure 4) indicates which kind of step was performed. **Failure** and **Success**
 164 are returned for trees that satisfy, respectively, the *failed* and *success* tests. **CutBrothers**
 165 indicates the interpretation of a superficial cut: *step* was called on an *And*-layer and its
 166 *next_tree* is the cut atom. Finally, **Expanded** accounts for the interpretation of predicate calls
 167 and non-superficial cuts.

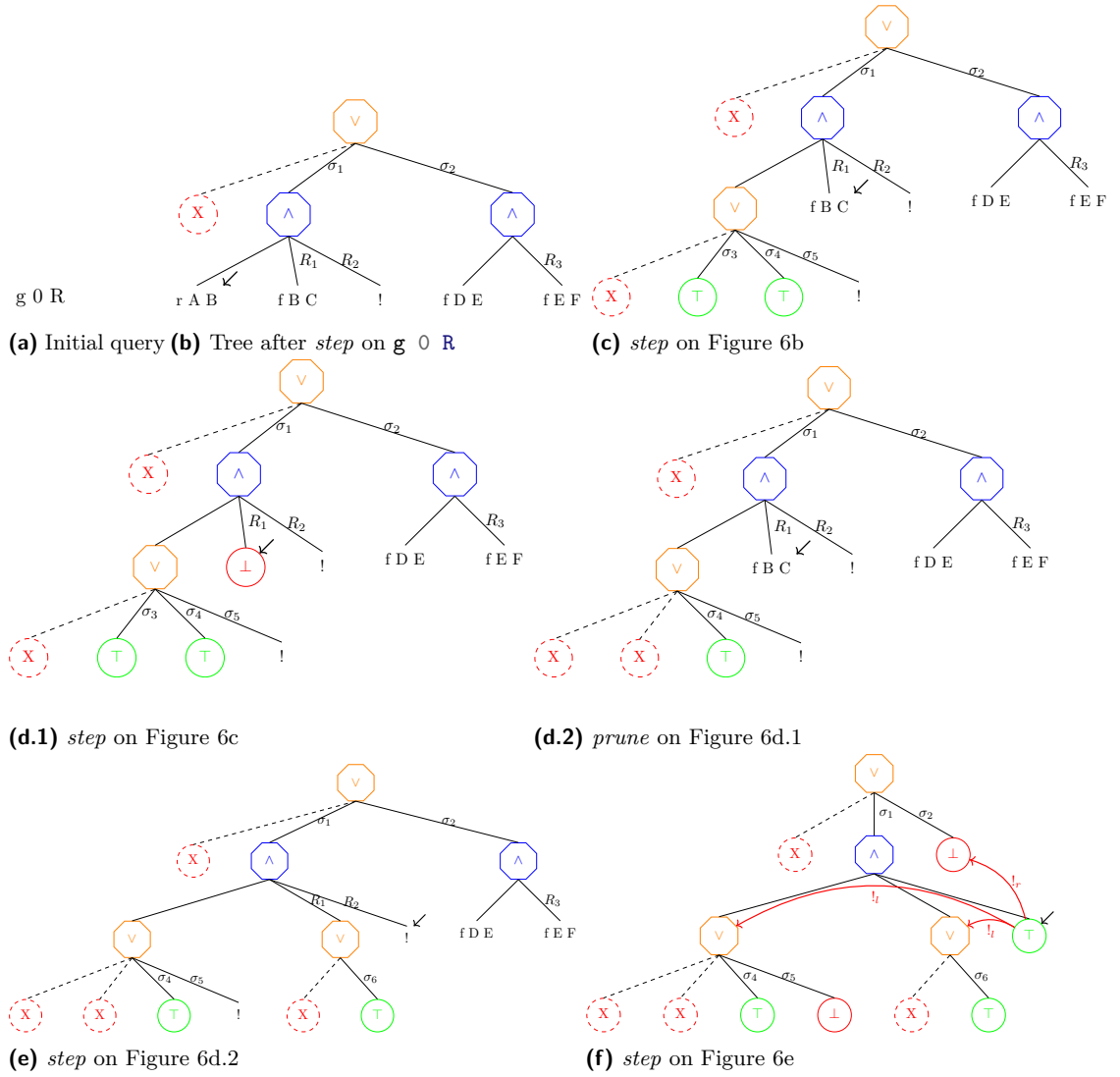
168 The *step* procedure evaluates atoms. In particular, it leaves the tree unchanged when the
 169 tree is *success* or *failed*.

170 **Lemma** succ_step_iff: $\forall p \ v \ \sigma \ t, \text{ success } t \leftrightarrow \text{ step } p \ v \ \sigma \ t = (v, \text{ Success}, t).$ (1)

171 **Lemma** fail_step_iff: $\forall p \ v \ \sigma \ t, \text{ failed } t \leftrightarrow \text{ step } p \ v \ \sigma \ t = (v, \text{ Failed}, t).$ (2)

172 *step* expands *Todo* nodes by calling backchain, which returns a list *l* of alternatives. If *l* is
 173 empty, then the current call is replaced by *KO*. Otherwise, it is replaced by a right-skewed

¹ The substitutions $\sigma_1 \dots \sigma_6$ are from Section 2.1. R_1, R_2 , and R_3 are reset points and correspond respectively to the lists of atoms [f Y Z, !], [!], and [f Y Z].



■ **Figure 6** Some tree representations¹

tree of *Or* nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is *OK*; otherwise, it is a right-skewed tree of *And* nodes.

Figure 6b depicts the tree corresponding to the running example (Section 2.1) as it undergoes calls to *step*. We run query $q \ 0 \ R$ against the program P in Figure 1. Let c_0 , c_1 , and c_2 be the children of the root. By construction, c_0 is the $\square$ tree, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P . We need the initial $\square$ subtree so that we can attach a substitution to c_1 . In particular, c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in Section 2.1. Reset points are marked with the R symbol: $R_1 := [f \ Y \ Z, !]$, $R_2 := [!]$, and $R_3 := f \ Y \ Z$. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of *step* triggering backchaining appear in Figures 6c–6e.

3.2.1 The interpretation of a cut

The step corresponding to a cut performs two actions: (i) the cut node is replaced by *OK*; and (ii) the subtrees in the scope of *Cut* are pruned. More precisely, (ii) prunes the left siblings of the *Cut* node (in the same *And*-layer, denoted $!_l$) and prunes the right uncles of *Cut* (in the one-level-up *Or*-layer, denoted $!_r$).

An example of the impact of cut is shown in Figure 6f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. We note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by *KO*. This amounts to discarding the third rule of predicate r (see Section 2.1).

3.3 The prune procedure

When backchain returns an empty list, *step* produces a *failed* tree and the computation must backtrack. The *prune* procedure is responsible for producing the new tree from which the computation can continue.

In Figure 6d.1 we encounter a failure because no rule applies to the atom $f \ B \ C$ under substitution σ_3 , which maps B to 0. The *prune* procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the *OK* node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the *KO* node with the information stored in the reset point R_1 . This restores the call $f \ B \ C$, which can then be reconsidered under substitution σ_4 .

Furthermore, *prune* serves another important purpose. Its behavior depends on the boolean flag it receives as input: *prune* $\perp A$ recursively removes all failures (if any) from A , whereas *prune* $\top A$ removes the first success in A . For example, the tree in Figure 6f contains a successful path that maps R to 2. Calling *prune* on this tree returns $\square$, since there are no alternatives in the tree after the success.

Some interesting properties of *prune* are shown below; they illustrate how *prune* complements *step* with respect to failed, success, and *path_atom* trees.

Lemma incomplete_prune_id: $\forall b \ t, \text{incomplete } t \rightarrow \text{prune } b \ t = \bullet t.$ (3)

Lemma prune_failedF: $\forall b \ t \ t', \text{prune } b \ t = \bullet t' \rightarrow \text{failed } t' = \perp.$ (4)

Lemma pruneFN_fail: $\forall t, \text{prune } \perp t = \square \rightarrow \text{failed } t.$ (5)

sottolineare le
sostituzioni

216 3.4 The *runT* inductive

217 The inductive relation *runT* relates four inputs to three outputs. The inputs are a program,
 218 a set of variables, an initial substitution σ , and a tree t . The outputs are (i) an optional
 219 result r of the form $\bullet(\sigma', t')$, (ii) a boolean b , and (iii) an updated set of variable names.

220 The main output is r : if $r = \square$, interpretation fails; otherwise σ' is the computed solution
 221 substitution and t' represents the remaining, not-yet-explored alternatives. The boolean b
 222 records whether a superficial cut was evaluated during execution, and the updated variable
 223 set tracks freshly generated names.

224 *runT* has four cases, depicted as inference rules in Figure 7. (STOPT) If *next_tree* t is a
 225 success, then the substitution σ' is extracted from t (starting from σ) and the input tree is
 226 cleaned of its successful path. (STEPT) If *next_tree* t is an atom, then *step* is invoked to
 227 evaluate t , and *runT* is called recursively on the updated tree. (BACKT) If *next_tree* t is a
 228 failure, then *prune* is called to clear the failed path; if the resulting cleaned tree exists, *runT*
 229 is called recursively on it. Finally, (FAILT) accounts for trees with no solutions.

230 These rules are deterministic:

$$231 \text{ Lemma runT_det: } \forall p \ v_0 \ \sigma \ t \ r \ r' \ b \ b' \ v \ v', \quad \text{runT } p \ v_0 \ \sigma \ t \ r \ b \ v \rightarrow \text{runT } p \ v_0 \ \sigma \ t \ r' \ b' \ v' \rightarrow r' = r \wedge v' = v \wedge b' = b. \quad (6)$$

232 The proof is by induction on the first *runT* derivation and is immediate, because the
 233 rules are selected based on *next_tree* t . In particular, the hypotheses *success* t , *path_atom* t ,
 234 and *failed* t are mutually exclusive. Moreover, by Lemma 6, the hypothesis of FAILT implies
 235 *failed* t ; hence FAILT is exclusive with STOPT and STEPT, and it is also exclusive with
 236 BACKT since they impose different requirements on the output of *prune*.

$$\begin{array}{c} \frac{\text{success } t \quad \text{next_subst } \sigma \ t = \sigma' \quad \text{prune } \top \ t = t'}{\text{runT } v \ \sigma \ t \ \bullet(\sigma', t') \ \perp \ v} \text{STOPT} \\[10pt] \frac{\text{incomplete } t \quad b' = \text{is_cb } \text{tg} \ || \ b \quad \text{step } p \ v \ \sigma \ t = (v', \text{tg}, t') \quad \text{runT } v' \ \sigma \ t' \ r \ b \ v''}{\text{runT } v \ \sigma \ t \ r \ b' \ v''} \text{STEPT} \\[10pt] \frac{\text{failed } t \quad \text{prune } \perp \ t = \bullet t' \quad \text{runT } v \ \sigma \ t' \ r \ n \ v'}{\text{runT } v \ \sigma \ t \ r \ n \ v'} \text{BACKT} \\[10pt] \frac{\text{prune } \perp \ t = \square}{\text{runT } v \ \sigma \ t \ \square \ \perp \ v} \text{FAILT} \end{array}$$

■ **Figure 7** Rule system for *runT*

237 3.5 Properties of *runT*

238 Working with *runT* allows to prove some interesting properties about the behavior of the
 239 cut operator wrt disjunction, i.e. the *Or* node. We have proven several properties that are
 240 available at [this link](#). Below we show two of these lemmas.

$$241 \text{ Lemma runSST_or: } \forall p \ v \ v' \ \sigma \ \sigma' \ l \ l' \ \sigma_1 \ r, \quad \text{runT } p \ v \ \sigma \ l \ \bullet(\sigma', \bullet l') \ \top \ v' \rightarrow \text{runT } p \ v \ \sigma \ (\text{Or } \bullet l \ \sigma_1 \ r) \ \bullet(\sigma', \bullet(\text{Or } \bullet l' \ \sigma_1 \ \text{K0})) \ \perp \ v'. \quad (7)$$

242 **Lemma** `runNT_or`: $\forall p \ v \ v' \ \sigma \ t \ \sigma_1 \ t',$
`runT` $p \ v \ \sigma \ t \ \square \top \ v' \rightarrow$
`runT` $p \ v \ \sigma \ (Or \bullet t \ \sigma_1 \ t') \ \square \perp \ v'.$ (8)

243 **Lemma** `run_orSST` $p \ v \ v' \ \sigma \ \sigma' \ \sigma_1 \ l \ l' \ r \ r':$
`runT` $p \ v \ \sigma \ (Or \bullet l \ \sigma_1 \ r) \bullet(\sigma', \bullet(Or \bullet l' \ \sigma_1 \ r')) \perp \ v' \rightarrow$
 $\exists b, \text{runT } p \ v \ \sigma \ l \bullet(\sigma', \bullet l') \ b \ v' \wedge r' = \text{if } b \text{ then } KO \text{ else } r.$ (9)

244 In Lemma 7, we evaluate the tree l . This evaluation produces a success with signature σ'
 245 and a new tree l' . During the evaluation, a superficial cut in l is traversed. This implies that
 246 the evaluation of $\bullet l \vee_{\sigma_1} r$ returns σ' as substitution, and the alternatives are $\bullet l' \vee_{\sigma_1} KO$. The
 247 right-hand side of the *Or* node is replaced with *KO* by $!_r$.

248 In Lemma 8, we are in a similar scenario: t evaluates a superficial cut but, this time,
 249 produces a failure. This implies that the evaluation of $\bullet t \vee_{\sigma_1} t'$ also fails.

250 Finally, in Lemma 9, we evaluate $\bullet l \vee_{\sigma_1} r$. The evaluation succeeds, producing the result
 251 $\bullet(s_1, \bullet l' \vee_{\sigma_1} r')$. This means that l still has l' as unexplored alternatives. We deduce that
 252 the evaluation of l produces the substitution σ_1 with alternatives l' . However, we have no
 253 information about whether a superficial cut has been crossed in l . If this is the case, we
 254 know that r' is cut away by $!_r$; otherwise, r' has not been explored and is equal to r .

255 From these lemmas, it is clear how the order in which alternatives are evaluated changes
 256 the behavior of the program.

se riesco success
 (A or B) = suc-
 cess (B or A) if
 no sup cut in A
 and B

257 4 Stack semantics

258 We now introduce the ELPI semantics. The interpreter is close to that of the ELPI language
 259 and provides an operational semantics that is similar to the one proposed by Pusch in [18],
 260 which is in turn closely related to the semantics given by Debray and Mishra in [6, Section 4.3].
 261 Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that
 262 are also present in the Warren Abstract Machine [23, 1].

263 We represent a state of the ELPI language thanks to the two mutual inductives shown
 264 below.

Inductive `alts` $:= \text{nilA} \mid \text{consA of } (\Sigma * \text{goals}) \ \& \ \text{alts}$
with `goals` $:= \text{nilG} \mid \text{consG of } (A * \text{alts}) \ \& \ \text{goals}.$

265 An ELPI state is an enhanced two-dimensional list. The outermost list represents a list
 266 of alternatives in disjunction, each accompanied by the substitution that should be used
 267 for their interpretation. The innermost list is a list of atoms, that is, the list of goals in
 268 conjunction. These goals are decorated with a pointer to an alternative list, which is used to
 269 keep track of where to jump when a cut is interpreted. We call these special alternatives the
 270 *cut-to* alternatives.

271 The ELPI interpreter is called *runE* and has the following type.

272 **Inductive** `runS` $: \mathcal{F}_v \rightarrow \text{alts} \rightarrow \text{option } (\Sigma * \text{alts}) \rightarrow \text{Prop} :=$ (1)

273 The inductive *runE* represents a routine taking as input a set of variables and a list
 274 of alternatives and returning an option. $\square$ stands for interpretation with no solution,
 275 whereas $\bullet(\sigma, a)$ represent a successful interpretation with σ as solution and a as the list of
 276 non-yet-explored choice points.

277 The rule system for *runE* is given in Figure 8 and is made by 4 cases. We distinguish
 278 two main cases: if the list of alternatives is empty, (FAILE) we have a failure: we stop the
 279 computation and return $\square$. If the list of alternatives is the list $(\sigma_0, h) :: a$, the interpreter
 280 evaluates the first choice point h under the substitution σ_0 . If h is empty, (STOPE), then we

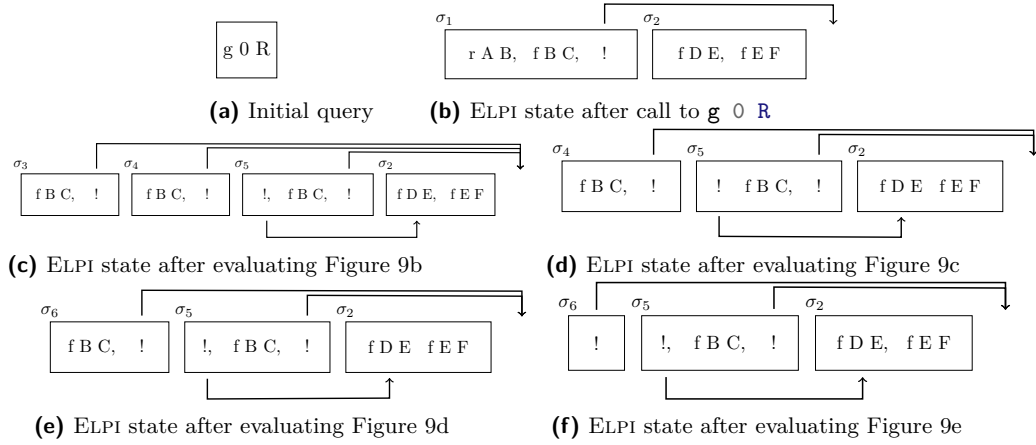
Dire che non ab-
 biamo le due
 info ritornate da
 runT

Definition $\text{stepE } v \ t \ \sigma \ a \ g :=$
 $\text{let } (v', r) := \text{bc } p \ v \ t \ \sigma \text{ in}$
 $(v', \text{save_alts } a \ g \ r).$

$$\frac{}{\text{runE } v \ ((\sigma, \emptyset) :: a) \bullet (\sigma, a)} \text{STOPE} \quad \frac{\text{runE } v \ ((\sigma, g) :: ca) \ r}{\text{runE } v \ ((\sigma, (\text{cut}, ca) :: g) :: _) \ r} \text{CUTE}$$

$$\frac{\text{stepE } v \ t \ \sigma \ a \ g = (v_1, a') \quad \text{runE } v_1 \ (a' ++ a) \ r}{\text{runE } v \ ((\sigma, (\text{call } t, _) :: g) :: a) \ r} \text{CALLE} \quad \frac{}{\text{runE } _ \ \emptyset \ \square} \text{FAILE}$$

■ **Figure 8** Rule system for runE



■ **Figure 9** Example of ELPI execution

have a success, i.e. we return the input σ_0 and leave a as the future choice points. Otherwise, h is the list $(t, ca) :: g$ where t is the first atom to evaluate, ca is its cut-alternative list and g is the list of future conjuncts. If t is a cut (CUTE), we need to keep evaluating g under σ_0 but we change the alternatives to ca . Finally, in the case of a call (CALLE), we backchain the call in the program. If rs is the list of rule premises returned by bc , then each list of premises if mapped to a list of goals (**goals**) have a as cut-alternative and the global list is translated to a list of alternative (**alts**).

Example of execution We propose an exemple of execution of an ELPI state in Figure 9. The arrows in the images represent the cut-alternative pointer of the cut. We have not put arrow going our from atom-calls since the interpreter ignores the cut-alternatives in this case (cf. CALLE).

Remark We want finally to point out how the lemma that we were able to prove in Section 3.5 are not easy to be expressed in the ELPI semantics. This is beacuse there is no sufficient structure in the ELPI state to understand what is the scope of the cut operator. For example, in an ELPI state made of these alternatives $a_0, a_1 \dots a_n$, we have no clear idea of what happens if the first alternative a_0 succeeds: there could be a cut in a_0 but, is this cut superficial? What is its impact wrt the alternatives $a_1 \dots a_n$? More then that, it is also possible that the state is hill-formed and the cut does not point to a suffix in the list of alternatives.

Dire una parola
 su backtracking
 = tl if bc = nil
 add subst to
 each alternative
 in Figure 9
 detail more?

```

300 Fixpoint valid_tree A :=
    match A with
    | Todo _ | OK | KO  $\Rightarrow$  T
    | Or  $\square$  _ B  $\Rightarrow$  valid_tree B
    | Or  $\bullet$ A _ B  $\Rightarrow$  valid_tree A &&
      ((B == KO) || B.base_or B)
    | And A B0 B  $\Rightarrow$  valid_tree A &&
      if success A then valid_tree B
      else B == big_and B0
    end.

301 Fixpoint t2l t  $\sigma$  (cp: alts) : alts :=
    match t with
    | OK  $\Rightarrow$  [( $\sigma$ ,  $\emptyset$ )]
    | KO  $\Rightarrow$   $\emptyset$ 
    | TA a  $\Rightarrow$  [( $\sigma$ , [(a,  $\emptyset$ )])]
    ...

```

(a) Valid tree

(b) Tree to list

■ **Figure 10** Valid tree and Tree to list

5 Equivalence of the two semantics

In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. In particular we define a translation from trees to stacks, called *t2l*. We do not explicitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [2].

Before describing the translation of trees to stacks (in Section 5.2) we need to define the notion of *valid tree*. The **tree** datatype indeed contains trees that cannot be generated by **runT** via **step** or **prune**, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

5.1 Valid trees (*valid_state*)

The class of valid trees is characterized by the function shown in Figure 10a.

While all base cases of tree are considered valid, we need to analyze carefully the cases for the *Or* and *And* constructors.

For the *Or* constructor, we distinguish two cases depending on whether the left hand side is present. If it is not present, then the right hand side must be a valid tree. Otherwise, the left hand side must be a valid tree and the right hand side must either be the *KO* tree or be unexplored. The first case occurs when the right hand side is cut out by the evaluation of a superficial cut present in the left hand side. In the latter case we use the **base_or** predicate to check that **B** is the right-skewed tree formed by a disjunction of conjunctions that is generated after a successful backchain to a **call**.

For the *And* constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. **B** must be the right-skewed tree containing conjunctions of premises atoms. This is enforced using the **big_and** produce that generates a tree out of the reset point B_0 , the same procude used to transform each alternative output by **bc** into the And-layer of the resulting tree. When the left hand side is successful, then the right hand side may have been explored, and consequently must be a valid tree.

5.2 From trees to stacks (*t2l*)

The *t2l* recursive function translates a tree to a stack. For space constraints Figure 10b only gives the base cases, while we describe the recursive cases in the following text. DAVVERO?

t	6a	6b	6c	6d.1 and 6d.2	6e	6f
$t2l\ t\ \varepsilon\ \emptyset$	9a	9b	9c	9d	9e	9f

■ **Table 1** Tree to list from Figure 6 and Figure 9

330 The function $t2l$ takes the tree t_0 , a substitution σ_0 and a list of choice points cp . These
 331 choice points represent alternatives that appears at the same level of the current tree, such
 332 as, for example, the right-siblings of a *Or* node. The substitution **TODO**.

333 The *OK* node represent one succesful alternative, thereofre it is translated into a singleton
 334 list with the empty list of goals. The *KO* node represent a failure, i.e. it is mapped to
 335 the empty list of alternatives. Finally, atoms are mapped to one alternative containing a
 336 single goal with the empty cut-to alternative list. They cannot point to cp since they are not
 337 right-uncle subtrees, i.e. **EXPLAIN**.

338 The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for
 339 B . The alternatives of B are valid choice points for A and should be passed to the translation
 340 of A . The two list of alternatives can be concatenated and, each atom, should add cp as
 341 new cut-to alternatives: A and B are one level lower wrt cp and therefore the cut they have
 342 should point to cp .

343 The translation of $A \wedge_{B_0} B$ starts with the translation of A . If the translation of A
 344 is an empty list we return the empty list of alternatives without even touching B nor B_0 ,
 345 since an empty translation for A indicates failure and this in turn implies the failure of the
 346 entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$ the B node represent the
 347 continuation of the first alternative g , whereas B_0 is the continuation of each alternative in a .

5.2.1 Example of $t2l$

348
 349 In Table 1 we point out the result of calling $t2l$ on the trees in Figure 6. The first raw
 350 contains the image reference of then trees and the second raw contains the reference of the
 351 images from the ELPI state in Figure 9. For the 3rd column in the table, we see that $t2l$ is
 352 not injective. There are different trees that maps to the same ELPI state. In fact, there are
 353 transitions in $runT$ that are not visible in $runE$.

5.3 Equivalence proof

354 The tree-based semantics exposes more information so to be usable to express the lemmas in
 355 Section 3.5. For the equivalence this information is irrelevant, hence define a version of $runT$
 356 that hides it.

Definition $runT'$ $p\ v\ \sigma\ t\ r := \exists\ b\ v',\ runT\ p\ v\ \sigma\ t\ r\ b\ v'$. (2)

357 Before going to the equivalence proofs, we want to claim a key lemma allowing to ignore
 358 the silent transition in the $runT$ inductive.

► **Lemma 1** (xxx).

Lemma $pruneF_t2l$: $\forall\ t\ t'\ b,$
 $valid_tree\ t \rightarrow failed\ t \rightarrow prune\ b\ t = \bullet t' \rightarrow$
 $\forall\ l\ \sigma,\ t2l\ t\ \sigma\ l = t2l\ t'\ \sigma\ l.$

Proof. By induction on the tree t . ◀

363 The first direction of the equivalence states that the execution of a valid tree is matched
 364 by the execution of the stack obtain by translating the tree and moreover the unexplored
 365 alternatives returned as a tree match the ones returned as a stack.

366 ► **Theorem 2** (xxx).

```

Lemma tree_to_elpi:  $\forall p\ t\ \sigma\ r,$ 
  let  $v := \text{vars\_tree } t \text{ '/' } \text{vars\_sigma } \sigma$  in
  valid\_tree  $t \rightarrow$ 
    runT'  $p\ v\ \sigma\ t\ r \rightarrow$ 
      let  $a := t2l\ t\ \sigma\ \emptyset$  in
      let  $r' :=$  if  $r$  is  $\bullet(\sigma', t)$  then  $\bullet(\sigma', t2l\ (\text{odflt } KO\ t)\ \sigma\ \emptyset)$ 
        else  $\square$  in
      runS  $p\ v\ a\ r'$ .

```

367 **Proof.** We proceed by induction on the execution of *runT'*. There are 4 cases to be treated.
 368 The first case comes from **STOPT**, it deals with successful trees, a successful tree must to
 369 start with an empty list and therefore we conclude with **STOPE**. The case for **STEPT**, need
 370 to treat two cases: *step* has evaluated either a cut or a call. Depending on this we should use
 371 **CUTE** or **CALLE** and then the induction hypothesis. The case for **BACKT** is a silent case:
 372 it represents the non-injective behavior of *t2l*, but, thanks to Lemma 1 and the induction
 373 hypotheses the conclusion is proved. The last case comes from the derivation **FAILT**. It is
 374 easily proved using **FAILE** and the fact that if *prune* returns $\square$ when trying to parse failure
 375 in t , then *t2l* of t returns the empty list. ◀ english!

376 The converse direction is stated following [2]: instead of using a function to translate a
 377 stack into a tree it states that any tree that translates to the given stack has an equivalent
 378 execution, i.e. gives the same solution and returns a unexplored tree that translates to the
 379 unexplored stack.

380 ► **Theorem 3** (xxx).

```

Lemma elpi_to_tree  $p\ v\ a\ r :$ 
  runS  $p\ v\ a\ r \rightarrow$ 
   $\forall \sigma\ t, \text{valid\_tree } t \rightarrow t2l\ t\ \sigma\ \emptyset = a \rightarrow$ 
  if  $r$  is  $\bullet(\sigma', a')$  then
     $\exists t', \text{runT}'\ p\ v\ \sigma\ t\ \bullet(\sigma', t') \wedge t2l\ (\text{odflt } KO\ t')\ \sigma\ \emptyset = a'$ 
  else runT'  $p\ v\ \sigma\ t\ \square$ .

```

381 **Proof.** The proof of Theorem 3 is based on the idea explained in [2, Section 3.5.1] and
 382 depicted in Figure 11: we have a state t whose translation is a , we proceed by induction on
 383 the execution of *runE*. This gives us not only the output a' but that hypothesis that it exists
 384 a tree t' such that its translation to the ELPI state is a' . This proof strategy is indirectly
 385 providing a kind of *l2t* function by combining the execution of the *runE* and *t2l*. ◀

386 Stating the converse direction in a symmetric way would have requires a function *l2t*
 387 transforming an stack a to a tree t . Writing this function is far from trivial, since the
 388 structure of the tree is lost by save alts and run that intuitively keep the state into a big
 389 disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover one
 390 would need to define a notion of valid stack that is equally intricate.

METTERLA

391 6 Case study: determinacy analysis

392 An interesting application of the tree semantics is the determinacy checking that was added
 393 in version 3 of the ELPI language [11]. The checker is meant to receive the signature of the
 394 predicates declared by the user and then verify that the rules of a deterministic predicate
 395 will return at most one solution, we define this particular kind of predicate as functions.

Thanks to this static verification, the user can better control unexpected backtracking at compile time: if an incoherence is rilevata then a fatal error explain why the current program may break determinacy. A function behaves deterministically if two main conditions are respected: *mutual exclusion* guarantees that at most one rule can be successfully used on a given query, whereas *rule checking* ensure that each rule does not creates choice points, e.g. by calling non-deterministic predicates.

For example, in `??`, we can declare the following signatures to the predicates.

$$f : \text{int} \rightarrow_i \text{int} \rightarrow_o \text{func} \quad g : \text{int} \rightarrow_i \text{int} \rightarrow_o \text{func} \quad r : \text{int} \rightarrow_i \text{int} \rightarrow_o \text{pred}$$

We see that `f` is a function since its rules are mutual exclusive and have empty body and `g` is a function since `r1` is cutting `r2` if `r1` succeeds due to the cut. Moreover, both rules The signature of the program are stored in the its `sig` projection, and, as in [11] we need to extends the type of predicates so that we are able to distinguish between deterministic and non-deterministic propositions. For example, in `??`, we have ...

A program execution behaves deterministically if given a program, a substitution and a tree to the `runT` inductive, then either the execution fails, or, if it succeeds, then the tree of alternatives is $\square$. Below we define `is_det`.

Definition `is_det p σ v t :=`
 $\forall r, \text{runT}' \text{ p } v \sigma t r \rightarrow \text{if } r \text{ is } \bullet(_, x) \text{ then } x = \square \text{ else } r = \square.$

use coq syntax instead of elpi?

The theorem we want to prove is the following.

Lemma `det_check_call p σ t:`
`let v := vars_tm t `|` vars_sigma σ in`
`check_P p → tm_is_det p.(sig) t → is_det p σ v (Todo (call t)).`

With `check_P` being the function checking the program wrt mutual exclusion and rule checking and `tm_is_det` being the function testing if the term head refers to a rigid constant with deterministically type. The proof of this lemma should be done on the derivation of the program execution. However, in this definition, the induction hypothesis is too weak, since in we would have to prove ... but we will not be able to show that the ... is deterministic.

To solve the problem, we need to work on “deterministic trees” (`det_tree`), show that if a tree respect this condition, and then prove the following theorem which is more generic then `det_check_call`.

Lemma `det_check_tree σ v p t:`
`vars_tree t `<= v → vars_sigma σ `<= v →`
`check_P p → det_tree p.(sig) t → is_det p σ v t.`

Since `det_check_call` is an instance of `det_check_tree`, its proof is now trivial.

As an important remark, proving the same lemma with the stack semantics, it is difficult to correctly define the predicate `det_stack`, checking for the deterministic behavior of a generic stack, since, similar to Section 3.5, the lack of structure make this goal hard. We need therefore an intermediate data structure, like the tree to encompass this problem.

7 Related work

The input mode of Elpi, inspired by the XXXX of BProlog, has only an impact in the determinacy analysis that does not required to be preceeded by a mode analysis. Well moded program propagate the groundness from input arguments to output arguments. As a result

if the initial query is ground then unification degenerates to matching (for input arguments). Considering a semantics where input arguments are match has the following impact on det analysis: axiom XXX does not require q to be ground.

The only mechanization of Prolog's semantics we could find is the one by Pusch in ISABELLE/HOL [18] that is based on the model of Debray and Mishra [6, Sec. 4.3]. Their work start from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [23, 1]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in section XXX that pushed us to develop an more explicit semantics (with respect to cut). In some way our work is complementary, showing the stack based semantics equivalent to more optimized one, we could consider implement in Elpi.

Another relevant work are the ones by King cite, but we could not find their code. Their semantics is denotational and cannot easily cover all the features our paper XX does, but would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by XXX and we thank Y.Bertot for insightful discussions on the topic. In XXX Bertot relates bla bla, without never constructing a decompiler, exactly as we never build a list to tree function.

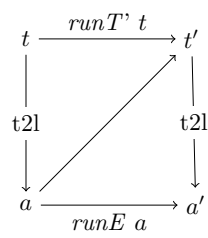
8 Conclusion

We do this. This is a first step for XXXX. We need to add substitutions. The missing part of the proof becomes related to abstract interpretation, since the det types for a lattice and one has to prove that the concrete runtime values variables take agree with the abstraction computed statically. This proof is ongoing and somehow orthogonal to the current one since the cut operator and the HO feature do not interfere with each other in complex ways.

References

- 1 Hassan Ait-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.
- 2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: <https://inria.hal.science/inria-00073199v1>.
- 3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.
- 4 Cyril Cohen, Enzo Crance, and Assia Mahboubi. TrocQ: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.
- 5 Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPICs*, pages 34:1–34:21, 2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPICs.FSCD.2020.34>, doi:10.4230/LIPICs.FSCD.2020.34.
- 6 Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:10.1016/0743-1066(88)90007-6.
- 7 Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λ Prolog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: <https://inria.hal.science/hal-01176856v1>, doi:10.1007/978-3-662-48899-7_32.

- 475 **8** Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq*
476 *Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- 477 **9** Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-
478 language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024.
479 doi:10.1145/3678232.3678233.
- 480 **10** Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic program-
481 ming language with cut. In *Practical Aspects of Declarative Languages: 28th International*
482 *Symposium, PADL 2026, Rennes, France, January 12–13, 2026, Proceedings*, pages 77–95,
483 Berlin, Heidelberg, 2026. Springer-Verlag. doi:10.1007/978-3-032-15981-6_5.
- 484 **11** Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic
485 programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects*
486 *of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.
- 487 **12** Benjamin Grégoire, Jean-Christophe Lécenet, and Enrico Tassi. Practical and sound equality
488 tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing
489 Machinery, 2023. doi:10.1145/3573105.3575683.
- 490 **13** Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory
491 in higher order constraint logic programming. In *Mathematical Structures in Computer*
492 *Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/
493 S0960129518000427.
- 494 **14** Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least
495 Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors,
496 *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leib-*
497 *niz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany,
498 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: [https://drops.dagstuhl.](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27)
499 [de/entities/document/10.4230/LIPIcs.ITP.2025.27](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27), doi:10.4230/LIPIcs.ITP.2025.27.
- 500 **15** math-comp. finmap — finite maps for mathcomp, 2026. URL: [https://github.com/](https://github.com/math-comp/finmap)
501 [math-comp/finmap](https://github.com/math-comp/finmap).
- 502 **16** Dale Miller. A logic programming language with lambda-abstraction, function variables, and
503 simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.
- 504 **17** Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge
505 University Press, 2012.
- 506 **18** Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris
507 Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors,
508 *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer
509 Berlin Heidelberg.
- 510 **19** SWI-Prolog Documentation. Predicate `!/0` — cut (built-in predicate), 2026. Accessed via SWI-
511 Prolog PlDoc reference. URL: https://www.swi-prolog.org/pldoc/doc_for?object=!/0.
- 512 **20** Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λProlog
513 dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January
514 2018. URL: <https://inria.hal.science/hal-01637063>.
- 515 **21** Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of
516 *LIPIcs*, pages 29:1–29:18, September 2019. URL: <https://inria.hal.science/hal-01897468>,
517 doi:10.4230/LIPIcs.CVIT.2016.23.
- 518 **22** Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi.
519 Master’s thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.
- 520 **23** David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309,
521 SRI International, Artificial Intelligence Center, Computer Science and Technology Division,
522 Menlo Park, CA, USA, October 1983. URL: [https://www.sri.com/wp-content/uploads/](https://www.sri.com/wp-content/uploads/2021/12/641.pdf)
523 [2021/12/641.pdf](https://www.sri.com/wp-content/uploads/2021/12/641.pdf).



■ Figure 11